

QuantumDx — Complete Project Context

Hybrid Quantum Machine Learning Platform for Early Cardiovascular Disease Detection

Smart India Hackathon 2026 — Problem Statement SIH26139

Handoff document for continuing project work across AI sessions

Generated: Phase 5 complete, awaiting Phase 6 (September 2026)

Abstract

This document is a complete, self-contained record of the QuantumDx project: what problem it solves, every design decision made, every phase completed so far (Phase 1 through Phase 5), the exact numerical results obtained, the engineering principles enforced throughout, the current repository structure, and the remaining phases still to be built. It is written so that a reader (human or AI assistant) with *zero prior context* can read it once and be immediately productive continuing the project — verifying claims against the repository, running the next phase, or reasoning about design trade-offs already settled. Every number quoted here comes from an actual executed run, not an estimate, unless explicitly marked otherwise.

Contents

1	Project Overview	3
1.1	The problem statement	3
1.2	Our product decision	3
1.3	The governing documents	3
1.4	The IBM Quantum hardware plan (already decided, not yet executed)	3
2	Engineering Principles Enforced Throughout	4
2.1	Leakage safety	4
2.2	Reproducibility	4
2.3	Honesty / anti-overclaiming	5
2.4	Reuse, not duplication	5
2.5	Documentation-in-sync-with-code	5
3	Phase 1 — Dataset Understanding and Validation	5
3.1	What was built	5
3.2	Key verified facts about the dataset	6
3.3	The 13 predictors and their semantic types	6
3.4	Important findings worth remembering	6
4	Phase 2 — Leakage-Proof Preprocessing & Feature Pipeline	7
4.1	What was built	7
4.2	Key configuration (PreprocessingConfig, defaults)	7
4.3	The locked split (referenced by every later phase)	7
4.4	Design decisions worth remembering	8
5	Phase 3 — Classical ML Baseline & Leakage-Safe Evaluation	8
5.1	Purpose	8
5.2	The four required models	8
5.3	Leakage-safe cross-validation architecture	8

5.4	The three experiments and their real results	9
5.5	Sanity check	9
6	Phase 4 — Quantum Data Pipeline & Fidelity Kernel (Infrastructure)	10
6.1	Purpose and scope	10
6.2	Critical environment decision: no <code>qiskit-aer</code> , no <code>qiskit-machine-learning</code>	10
6.3	The quantum data contract	10
6.4	The feature map	11
6.5	Backend abstraction	11
6.6	The fidelity kernel	11
6.7	Sanity experiment (20 train / 10 test subsample, seed 42) — real results	12
7	Phase 5 — Full-Scale QSVM Experiment	12
7.1	Purpose	12
7.2	Computational cost — measured before committing to the design	12
7.3	A key methodological question, resolved and verified	12
7.4	Hyperparameter tuning — full CV grid table (real run)	13
7.5	Final locked-test evaluation (touched exactly once)	13
7.6	Descriptive comparison with Phase 3 (no significance test performed, by design)	13
8	Current Repository Structure	14
8.1	Environment snapshot	15
9	How to Run Everything (command reference)	15
10	Future Phases	15
10.1	Phase 6 — Scoped IBM Quantum Hardware Validation (next, not started)	15
10.2	Later phases (per <code>MVP_SPEC.md</code> 's 13-phase plan, not yet started)	16
11	Key Numbers Quick-Reference	17

1 Project Overview

1.1 The problem statement

SIH26139 — Hybrid Quantum Machine Learning Platform for Early Disease Detection. The problem statement (Smart India Hackathon 2026) asks for a platform that integrates classical preprocessing and feature engineering with quantum-enhanced learning models (QSVM, QNN, or VQC), applied to biomedical data, supporting data ingestion, hybrid model training, prediction, explainability, and performance evaluation against purely classical baselines. Central requirements: a hybrid quantum-classical architecture, classical preprocessing, feature engineering/selection, quantum-enhanced ML, prediction, explainability, performance evaluation, comparison with classical baselines (accuracy, sensitivity, specificity), computational efficiency, generalization, scalability, quantum simulator compatibility, near-term quantum hardware compatibility, and comprehensive documentation.

1.2 Our product decision

- **Disease:** Cardiovascular disease.
- **Initial task:** Early cardiovascular disease risk detection (binary classification).
- **Development dataset:** UCI Heart Disease dataset (Cleveland subset), used explicitly as a *development/benchmark* dataset, not a claim about high-dimensional biomedical data in general. The architecture is deliberately designed so a larger cardiovascular dataset can be substituted later via configuration, not a rewrite.
- **Project name:** QuantumDx.

1.3 The governing documents

Two large planning documents were produced before any code was written, and both remain the source of truth for scope and philosophy:

- **PRD.md** — the full Product Requirements Document. 43 sections covering executive summary, problem statement, product goals/non-goals, personas, functional and non-functional requirements, the complete ML pipeline, classical and quantum requirements, explainability, benchmarking, robustness, dashboard/API/architecture, a SIH26139 requirement-to-feature traceability matrix, and acceptance criteria. **Never modified after creation.**
- **MVP_SPEC.md** — the MVP Specification and Implementation Plan. Defines the *smallest complete* system satisfying SIH26139: the exact dataset schema, the 13-phase development plan (Phase 0 through Phase 12), project structure, engineering practices, and a real-hardware validation plan (Section 8.5, described in §1.4 below). **Never modified after creation.**

Both documents establish, and every phase since has upheld, three non-negotiable rules:

1. **Depth loses to completeness.** A working end-to-end chain beats a deep but partial one.
2. **Every stage must be demonstrable.** If it cannot be shown, it is not done.
3. **Honesty is a feature.** No claim of quantum advantage without a confidence interval and, eventually, a significance test. A null or negative result, measured rigorously, is a valid and reportable outcome.

1.4 The IBM Quantum hardware plan (already decided, not yet executed)

The team has IBM Quantum cloud access (`qiskit-ibm-runtime==0.44.0` is already installed). The plan, fixed in **MVP_SPEC.md** §8.5 and unchanged since:

- The *full* classical-vs-quantum benchmark stays on the simulator, permanently. The full training kernel is $\approx 43,900$ circuit evaluations $\approx 3\text{--}4$ hours of QPU time, against a free-tier allowance on the order of 10 minutes/month — roughly $20\text{--}40\times$ over budget for a single run, before even considering repeated CV or multiple seeds.
- A **separate, small, scoped hardware validation experiment** is planned for Phase 6: a fixed 30-train/10-test subsample (≈ 735 circuits, $\approx 3\text{--}4$ minutes of QPU time), with the identical small kernel computed three ways — exact statevector simulator, a noisy simulator, and the real IBM device — then compared via element-wise kernel divergence, diagonal-fidelity deviation (a ground-truth-free noise indicator, since $K(x, x)$ should be exactly 1), and kernel-concentration statistics.
- This hardware run is executed *once*, cached, and never queued live during a demo (queue times are unbounded).
- Hardware results are auditable (device name, calibration timestamp, job IDs, QPU seconds) but explicitly *not* claimed to be bit-reproducible, because device calibration drifts daily.

2 Engineering Principles Enforced Throughout

These principles were established early and have been mechanically enforced (via code structure and tests, not just stated) in every phase since. A reader picking up Phase 6 should preserve all of them.

2.1 Leakage safety

- The train/test split happens *before* any statistic is ever computed (Phase 2), and is never re-created by a later phase.
- Every fitted transformer (imputer, encoder, scaler, feature selector, PCA) is fit on the training partition only, and only ever `.transform()`-ed on test/validation data.
- Cross-validation refits the *entire* preprocessing chain inside every fold (Phase 3), using sklearn’s own `GridSearchCV` machinery rather than a hand-rolled loop.
- **Structural leakage guards:** tuning functions (`run_grid_search`, `run_qsvm_grid_search`) have *no test-set parameter in their signature at all* — verified directly via `inspect.signature()` in tests, not just by convention.
- **Behavioral leakage guards:** dedicated tests perturb an extreme, synthetic value into the test set and confirm training-derived statistics (imputer medians, scaler means, PCA axes, CV fold scores) do not change.
- The locked test set is touched by exactly one function, exactly once, after model selection is complete.

2.2 Reproducibility

- A fixed random seed (42) propagates through every stochastic component: the split, cross-validation folds, `mutual_info_classif`’s internal estimator, every classical model’s `random_state`, bootstrap resampling.
- A deterministic `split_id` (a short SHA-256-derived fingerprint of the split parameters) is computed once in Phase 2 and reproduced — never re-derived — by every subsequent phase. Its value for the default configuration is `e471025b07519a64`, and it appears verbatim in every phase’s results and documentation.
- Every experiment’s run record captures library versions, configuration, seeds, and timings as JSON.

- Immutable (`frozen=True`) configuration dataclasses (`PreprocessingConfig`, `QuantumConfig`) prevent one experiment’s config from silently mutating and contaminating another’s.

2.3 Honesty / anti-overclaiming

- No document claims quantum advantage, quantum superiority, or clinical validity anywhere.
- Every results document distinguishes *CV variability* (mean/std/t-distribution CI across folds, computed on training data only) from *final test uncertainty* (percentile bootstrap CI on the single locked-test evaluation) and never conflates the two.
- Model-selection criterion is fixed *before* results are observed (mean CV ROC-AUC, not accuracy) and is never changed retroactively to produce a better-looking result, even when a selected hyperparameter produces an ugly, degenerate outcome (this happened in Phase 5 — see §7).
- Every phase’s documentation has an explicit “What this does NOT establish” section.
- Raw data immutability is checksum-verified (SHA-256) after every phase’s code runs, not just claimed.

2.4 Reuse, not duplication

- Phase 3 (classical) reuses Phase 2’s preprocessing classes directly via thin sklearn-compatible adapter classes (`SharedFeatureTransformer`, `QuantumReadyTransformer`) rather than reimplementing preprocessing logic.
- Phase 4 (quantum infrastructure) reuses Phase 2’s `run_preprocessing_pipeline()` directly for its data contract — no new split, no new PCA fit.
- Phase 5 (QSVM) reuses Phase 3’s scoring dictionary (`CV_SCORING`), model-selection constant (`MODEL_SELECTION_METRIC`), and — critically — Phase 3’s *exact* `evaluate_on_test()` function for the final locked-test evaluation, unmodified. The same `ModelResult` dataclass is used for both classical and quantum models.
- Only one package (`src/quantum/`) is allowed to import Qiskit, so an API-breaking Qiskit upgrade has a confined blast radius.

2.5 Documentation-in-sync-with-code

Every phase produced a Markdown document in `docs/` that is explicitly declared to be *derived from* the code and results, with a stated rule: if a number in the document ever disagrees with a fresh run of the code, the code’s output is authoritative and the document must be corrected. Every non-trivial claim is paired with the specific test that verifies it (a “Traceability” table appears at the end of every phase document).

3 Phase 1 — Dataset Understanding and Validation

Status: **DONE**

Code: `src/data/inspect_dataset.py`

Docs: `docs/DATASET.md`

Tests: `tests/test_dataset.py` (27 tests)

Results: —

3.1 What was built

A read-only inspection and structural-validation module for `data/raw/processed.cleveland.data`. It loads the raw, headerless, comma-separated UCI file, assigns the 14 documented column names, treats “?” as the missing-value marker, and produces a full data-quality report: shape, dtypes,

missing-value counts, duplicate detection, numeric/categorical profiles, domain-violation checks against the documented UCI value ranges, class distribution, and zero-variance-column detection. **No imputation, encoding, scaling, or target binarization happens in Phase 1** — it is inspection only.

3.2 Key verified facts about the dataset

Property	Value
Source file	<code>data/raw/processed.cleveland.data</code>
Rows / Columns	303 / 14 (13 predictors + target <code>num</code>)
SHA-256 checksum	<code>a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8</code>
Missing values	<code>ca</code> : 4 cells; <code>thal</code> : 2 cells; all others: 0
Duplicate rows	0
Raw target <code>num</code> distribution	0→164, 1→55, 2→36, 3→35, 4→13
Binary target preview (<code>num>0</code>)	0 (no disease): 164 (54.1%); 1 (at risk): 139 (45.9%)
Categorical domain violations	0 (every observed value matches the documented UCI codes)

3.3 The 13 predictors and their semantic types

Group	Features	Notes
Continuous numeric	<code>age</code> , <code>trestbps</code> , <code>chol</code> , <code>thalach</code> , <code>oldpeak</code>	Median-imputed later, then scaled
Discrete numeric (ordinal count)	<code>ca</code> (0–3 vessels)	Kept numeric, not one-hot, to preserve order
Ordinal categorical	<code>slope</code> (3 levels)	Mode-imputed, kept as an ordered code
Binary categorical	<code>sex</code> , <code>fbs</code> , <code>exang</code>	Already 0/1; imputed only, never encoded/scaled
Nominal categorical	<code>cp</code> , <code>restecg</code> , <code>thal</code>	One-hot encoded after imputation

3.4 Important findings worth remembering

- **The UCI documentation itself is inconsistent:** `heart-disease.names` states missing values are marked `-9.0`, but the actual processed file uses `"?"`. This was verified empirically (`-9.0` never appears; `"?"` appears exactly 6 times) and the loader trusts the empirical evidence, not the stale documentation.
- **`chol=0` is a known undeclared missing-value sentinel at other UCI sites**, not Cleveland: verified that `chol=0` occurs in 100% of the 123 Switzerland records and 24.5% of the 200 VA records, but *zero* times in Cleveland. This directly motivated Phase 2’s decision to declare sentinel codes per-column, per-dataset, in configuration rather than hardcoding them.
- **The clinical-leakage concern about `ca/thal`:** both are results of specialist cardiac investigations (fluoroscopy, thallium stress test) typically ordered once disease is already suspected. `ca` in particular sits close to a direct read-out of the angiographic diagnosis. This was flagged in Phase 1 and became the basis for Phase 2/3’s “screening” feature-set ablation (features minus `ca/thal`), which Phase 3 confirmed empirically matters (see §5).
- The inspection script was verified to *generalize* correctly: running it against the Switzerland file independently rediscovered the `chol=0` pattern and a 6.5% minority-class imbalance warning, proving the logic is genuinely dataset-driven, not overfit to Cleveland’s specific cleanliness.

4 Phase 2 — Leakage-Proof Preprocessing & Feature Pipeline

Status: **DONE**

Code: src/preprocessing/{config,pipeline,validation}.py

Docs: docs/PREPROCESSING.md

Tests: tests/test_preprocessing.py (41 tests)

Results: —

4.1 What was built

A pipeline that turns the raw dataset into **two** feature representations sharing **one** upstream stage, so the two branches can never silently diverge in how the data was cleaned:

```
RAW DATA
-> TARGET CREATION (num -> target; target = 1 if num > 0 else 0; num preserved)
-> STRATIFIED TRAIN/TEST SPLIT (80/20, seed 42) <- split BEFORE any fitting
-> PREPROCESSING (impute -> encode -> scale) [fit on TRAIN only]
-> FEATURE SELECTION (SelectKBest, mutual_info) [fit on TRAIN only]
    |-----|
    v v
CLASSICAL-READY PCA (4 comps) + range-normalize to [0,pi] [fit on TRAIN only]
FEATURES (10 cols) |
                    v
                    QUANTUM-READY FEATURES (4 cols, in [0, pi])
```

4.2 Key configuration (PreprocessingConfig, defaults)

Field	Default
random_seed	42
test_size / stratify	0.20 / True
numeric_impute_strategy	median
categorical_impute_strategy	most_frequent
scaler	standard (also supports minmax, robust)
feature_selection_method	mutual_info (also supports f_classif)
feature_selection_k	10
pca_n_components	4 — this fixes the quantum branch’s qubit count for all later phases
quantum_range	[0, π]
feature_set	full (also supports “screening”: excludes ca, thal)

4.3 The locked split (referenced by every later phase)

Property	Value
Train / Test rows	242 / 61
split_id	e471025b07519a64
Train class balance	0: 131 (54.1%), 1: 111 (45.9%)
Test class balance	0: 33 (54.1%), 1: 28 (45.9%)
Features after impute+encode+scale (pre-selection)	17 (full feature set)
Selected features (top-10 by mutual information)	chol, thalach, ca, slope, sex, exang, cp_2.0, cp_4.0, thal_6.0, thal_7.0
PCA cumulative explained variance (4 components)	82.0%

4.4 Design decisions worth remembering

- **MinMaxScaler(`clip=True`) for the quantum range.** Discovered during development: a test-set PCA value can legitimately fall slightly outside the training-derived $[0, \pi]$ range (expected, since the scaler is fit on train only). `clip=True` guarantees the quantum-ready output is always bounded, without ever using test statistics to derive the bound.
- **SharedFeaturePipeline** is a hand-written two-stage object (not a bare sklearn `Pipeline`), because `feature_selection_k` must be clipped against the *actual* post-one-hot-encoding column count, which is only known after the encoder has been fit.
- **The screening ablation** (`feature_set="screening"`) is fully wired and tested but was *not* run as an experiment in Phase 2 — only prepared, per the phase’s explicit scope. Phase 3 later ran it.
- **The dedicated leakage test** constructs a synthetic extreme value (cholesterol = 999,999) in the test set and proves the training-fitted scaler mean, imputer median, and PCA axes are bit-identical whether or not that value exists, because the test set is never part of any `.fit()` call.

5 Phase 3 — Classical ML Baseline & Leakage-Safe Evaluation

Status: **DONE**

Code: `src/classical/{models,tuning,evaluation,train_baselines}.py`

Docs: `docs/CLASSICAL_BASELINE.md`

Tests: `tests/test_classical.py` (20 tests)

Results: `results/classical/**`

5.1 Purpose

Establish a rigorous, fairly-tuned classical benchmark that later quantum models must be compared against. **Explicitly not** an attempt to show classical ML is weak.

5.2 The four required models

Model	Library	Tuned parameter(s)
Logistic Regression	<code>sklearn.linear_model.LogisticRegression</code>	$C \in \{0.001, \dots, 100\}$ (6 values)
RBF-Kernel SVM	<code>sklearn.svm.SVC(kernel='rbf')</code>	C (4), γ (4) — 16 combos
Random Forest	<code>sklearn.ensemble.RandomForestClassifier</code>	$n_estimators$, max_depth , $min_samples_split$, $max_features$ (24 combos)
XGBoost	<code>xgboost.XGBClassifier</code> (v3.2.0)	$n_estimators$, max_depth , $learning_rate$, $subsample$, $colsample$ (48 combos)

A real sklearn-version bug was found and fixed, not hidden. Installed `scikit-learn==1.8.0` deprecates the `penalty` keyword on `LogisticRegression` in favor of a unified `l1_ratio` API; combining `penalty='l1'` with `solver='liblinear'` emitted an “inconsistent values” warning suggesting the `l1` request might not be honored correctly. The fix: tune C only, with the standard L2 penalty via the `lbfgs` solver — documented in code and in `CLASSICAL_BASELINE.md` §7.2, not silently patched over.

5.3 Leakage-safe cross-validation architecture

Two thin sklearn-compatible adapter classes wrap Phase 2’s preprocessing so it can be dropped into `GridSearchCV`:

- **SharedFeatureTransformer** — delegates to Phase 2’s `SharedFeaturePipeline`.

- `QuantumReadyTransformer` — delegates to Phase 2’s `build_quantum_pipeline` (PCA + range-normalize).

Both are placed as the first step(s) of an sklearn `Pipeline` together with the classifier, and the *whole pipeline* (preprocessing included) is handed to `GridSearchCV(cv=StratifiedKFold(5, shuffle=True, seed=42))`. sklearn’s own cross-validation machinery refits preprocessing fresh inside every fold. Model selection uses **mean CV ROC-AUC** (`MODEL_SELECTION_METRIC = "roc_auc"`), never accuracy.

5.4 The three experiments and their real results

Experiment 1 — full feature set, classical-ready (10 selected features):

Model	CV ROC-AUC	Test ROC-AUC	Test Sens.	Test Spec.
Logistic Regression	0.9019 $\pm$ 0.0130	0.9481	0.9286	0.8485
RBF-Kernel SVM	0.9055 $\pm$ 0.0189	0.9405	0.9286	0.8182
Random Forest	0.8894 $\pm$ 0.0193	0.9470	0.8214	0.9394
XGBoost	0.8871 $\pm$ 0.0216	0.9405	0.8571	0.8485

By the CV criterion, RBF-SVM ranks first (0.9055), narrowly ahead of Logistic Regression (0.9019) — *their 95% CIs overlap substantially, this is not a decisive win*. On the single locked test set, Logistic Regression scores highest (0.9481). This CV-vs-test disagreement is reported honestly as the reason CV, not the single 61-row test score, is trusted as the primary model-selection evidence.

Experiment 2 — screening (11 features, `ca/thal` excluded), classical-ready: Every single one of the four models’ CV ROC-AUC *dropped* when `ca/thal` were removed, consistently, by roughly 5 percentage points (mean across models: 0.896 $\rightarrow$ 0.845). This uniform effect across four architecturally different models is strong evidence that `ca/thal` genuinely carry substantial predictive signal beyond the other 11 features — confirming the Phase 1 clinical-leakage concern was well-founded. `ca/thal` were never removed from the primary dataset; this is a config-only ablation.

Experiment 3 — full feature set, quantum-ready (4 PCA components): This is the benchmark a future quantum model must be compared against fairly, because it uses the identical 4-dimensional representation a quantum model will see.

Model	CV ROC-AUC (4-dim)	Test ROC-AUC
Logistic Regression	0.8610 $\pm$ 0.0336	0.9210
RBF-Kernel SVM	0.8604 $\pm$ 0.0377	0.9242
Random Forest	0.8486 $\pm$ 0.0482	0.9167
XGBoost	0.8616 $\pm$ 0.0367	0.9275
Mean	0.8579	—

Compressing to 4 PCA components (82.0% retained variance) costs ≈ 3.8 ROC-AUC points on average versus the full 10-feature representation — a real, expected, disclosed cost of dimensionality reduction.

Class-weighting side-check (Experiment 1’s tuned hyperparameters, re-evaluated with `class_weight='balanced'` / `scale_pos_weight`): negligible effect on ROC-AUC (all four models changed by <0.002 , well inside CV noise), with the expected small sensitivity-up/specificity-down trade-off. **Conclusion: class weighting is not used**, since the dataset is near-balanced (54.1%/45.9%) and weighting demonstrably does not help — checked empirically, not assumed.

5.5 Sanity check

Every classical ROC-AUC lands well below 1.0 (0.887–0.906 CV, 0.94–0.95 test) — plausible for this literature-standard dataset and *not* suspiciously perfect, which is itself a basic sanity check against

leakage.

6 Phase 4 — Quantum Data Pipeline & Fidelity Kernel (Infrastructure)

Status: **DONE**

Code:

`src/quantum/{config,data_contract,feature_maps,backends,kernel,sanity_experiment}.py`

Docs: `docs/QUANTUM_PIPELINE.md`

Tests: `tests/test_quantum.py` (53 Phase-4 tests)

Results: `results/quantum/sanity/`

6.1 Purpose and scope

Build the infrastructure that turns an already-preprocessed patient record into a quantum kernel value — deterministically, reproducibly, without touching the test set in any way that matters. **Explicitly not** a benchmark: no QSVM was trained, no accuracy/sensitivity/specificity was reported, nothing was executed on real IBM hardware.

6.2 Critical environment decision: no qiskit-aer, no qiskit-machine-learning

The installed `qiskit` version is **2.2.3** (a major, recent version). Installing `qiskit-aer` or `qiskit-machine-learning` would have pulled in `numpy>=2.0` — a major version jump from the `numpy==1.26.4` that Phases 1–3’s tested pipeline (`pandas`, `scikit-learn`, `xgboost`) runs on, verified via `pip install -dry-run`. **Decision: use only core Qiskit.** The exact statevector simulator, `qiskit.quantum_info.Statevector.from_instruction()`, needs no Aer at all — at 4–6 qubits a statevector has 16–64 complex amplitudes, trivially cheap regardless of backend. This was verified by re-running the full Phase 1–3 test suite (68 tests) after adding Phase 4 code and confirming zero regressions and an unchanged `numpy` version.

A real API-deprecation issue was found and worked around, not hidden. `qiskit==2.2.3` deprecates the class-based `ZZFeatureMap`/`ZFeatureMap`/`PauliFeatureMap` (removal planned for Qiskit 3.0) in favor of function-based replacements: `zz_feature_map`, `z_feature_map`, `pauli_feature_map`. This implementation uses the function-based API throughout.

6.3 The quantum data contract

`load_quantum_dataset()` calls Phase 2’s `run_preprocessing_pipeline()` directly — the same function Phase 3’s Experiment 3 used. **No new split is ever created.** Returns `x_train_quantum` (242×4), `x_test_quantum` (61×4), `y_train`, `y_test`, with `split_id` reproduced (not re-derived) as `e471025b07519a64`. `n_qubits` is derived from `PreprocessingConfig.pca_n_components` (never hard-coded), so the quantum pipeline can never silently disagree with Phase 2.

6.4 The feature map

Property	Value
Feature map	<code>zz_feature_map</code> (default; <code>z_feature_map</code> , <code>pauli_feature_map</code> also supported)
Qubits	4 (= <code>pca_n_components</code>)
Repetitions	2
Entanglement	linear (circular, full also supported)
Encoding	Angle encoding, one classical feature per qubit
Trainable quantum parameters	Zero — the map is a fixed template; only the 4 data values are bound in
Free parameters (data)	4
Logical circuit depth (decomposed)	19
Gate counts (decomposed)	{u: 22, cx: 12}

Why `zz_feature_map`: entangling (RZZ-family) terms give the kernel access to pairwise feature correlations, unlike `z_feature_map` (no entanglement at all, essentially a product kernel). **Why linear entanglement:** fewest two-qubit gates, cheapest to simulate, and most compatible with a real device’s limited connectivity (relevant for Phase 6, even though Phase 4 does not execute on hardware). **This configuration was deliberately not optimized or swept** — a defensible standard starting point, not a validated best choice.

6.5 Backend abstraction

```
QuantumBackend (Protocol: compute_statevector(), capabilities())
+-- StatevectorBackend IMPLEMENTED -- core Qiskit only, exact, deterministic
+-- NoisySimulatorBackend interface only -- raises NotImplementedError; a Phase 5/6
    decision
+-- IBMHardwareBackend interface only -- raises NotImplementedError; scoped to Phase 6
```

Backend selection is purely config-driven (`QuantumConfig.backend_name`); `kernel.py` never branches on the backend name itself.

6.6 The fidelity kernel

$$K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$$

$$|\phi(x)\rangle = U_\phi(x)|0000\rangle$$

Implemented efficiently: `compute_statevectors(X, feature_map, backend)` embeds every row *once* (a statevector depends only on its own row), giving $O(n)$ circuit evaluations; `kernel_matrix_from_statevectors(...)` assembles the matrix from cheap inner products, exploiting symmetry (upper triangle only) when applicable, but **the diagonal is always computed explicitly, never hardcoded to 1.0** — a measured deviation from exactly 1.0 is itself a useful floating-point correctness signal.

6.7 Sanity experiment (20 train / 10 test subsample, seed 42) — real results

Item	Value
K_train_train	/ (20,20) / (10,20) / (10,10)
K_test_train	/
K_test_test shapes	
Min / Max kernel value	3.6e-05 / 1.0
Diagonal deviation from 1.0	3.55e-15 (machine precision)
Reproducibility	Bit-identical recompute, max diff = 0.0
Split ID	e471025b07519a64 (matches Phase 2/3)
Explicitly labelled "status": "sanity_check_only_not_a_benchmark" in its own JSON output.	

7 Phase 5 — Full-Scale QSVM Experiment

Status: **DONE**

Code: src/quantum/{qsvm,qsvm_experiment}.py

Docs: docs/QUANTUM_QSVM.md

Tests: 21 new tests in tests/test_quantum.py (74 total quantum tests)

Results: results/quantum/phase5/

7.1 Purpose

Train the first real QSVM: `sklearn.svm.SVC(kernel='precomputed')` fed the Phase 4 fidelity kernel, computed at full scale on all 242 training and 61 test records, tuned leakage-safely, evaluated once on the locked test set, and compared descriptively (no significance test yet) against Phase 3's Experiment 3.

7.2 Computational cost — measured before committing to the design

Step	Measured time
242 training-row statevectors	0.449 s
61 test-row statevectors	0.085 s
K_train_train assembly (242×242, 29,161 pairs)	0.091 s
K_test_train assembly (61×242, 14,762 pairs)	0.036 s
Total kernel computation	0.662 s
GridSearchCV (5 C values × 5 folds = 25 fits)	≈0.4 s

This measurement is exactly why fine-grained mid-computation checkpointing was *not* built: restartability is implemented at the level of caching the two final matrices to disk (keyed by feature-map config + `split_id`), which is more than sufficient given sub-second real cost. Train and test statevectors are each computed *exactly once* and reused for both matrices — `compute_kernel_matrix_cached()` from Phase 4 was deliberately *not* used naively (it would have recomputed the 242 training statevectors a second time).

7.3 A key methodological question, resolved and verified

Is it leakage-safe to tune `C` by slicing the *one* precomputed 242×242 kernel per CV fold, rather than recomputing a fold-specific kernel? **Yes** — because $K(x_i, x_j)$ is a fixed function of two feature vectors with zero fitted/learned state (the feature map is a template, not a model). This was *empirically verified*, not just argued: a test compares `sklearn.model_selection.cross_val_score` on the full precomputed kernel against a hand-written loop that manually slices `K[train_idx][:,train_idx] / K[val_idx][:,train_idx]` per fold — the scores match exactly.

One disclosed, bounded simplification: the 4-dimensional PCA representation was fit *once* on all 242 training rows (the single, locked Phase 2/4 contract), not refit per CV fold the way Phase 3’s classical preprocessing was. This is a mild, inherited, explicitly-disclosed simplification upstream of Phase 5 itself, expected to have negligible practical impact at this sample size, and consistent with Phase 5’s explicit instruction to reuse the locked representation unchanged.

7.4 Hyperparameter tuning — full CV grid table (real run)

C	CV ROC-AUC	CV Accuracy	CV Sensitivity	CV Specificity	Rank
0.01	0.6945 ± 0.0281	0.5413	0.0000	1.0000	1 (selected)
0.1	0.6945 ± 0.0285	0.5413	0.0000	1.0000	2
1.0	0.6942 ± 0.0306	0.6403	0.5755	0.6943	3
10.0	0.6629 ± 0.0737	0.6403	0.6032	0.6709	4
100.0	0.6461 ± 0.0949	0.6406	0.5858	0.6869	5

A finding reported precisely, not smoothed over: the selected C=0.01 has the (barely) highest CV ROC-AUC, tied within noise against C=0.1 and C=1.0, but **CV sensitivity = 0.0 and specificity = 1.0 at that C** — the model predicts the majority class for every validation patient, in every fold. ROC-AUC (threshold-independent) can still register moderately above chance from ranking quality alone, even while the 0.5-threshold prediction has collapsed to one class. **This was not “fixed” by picking a different C** — the selection criterion was fixed *before* this result was observed (inherited from Phase 3 for consistency), and switching it post-hoc would itself be a form of outcome-chasing.

7.5 Final locked-test evaluation (touched exactly once)

Metric	Value	95% Bootstrap CI
Accuracy	0.6721	[0.557, 0.787]
Sensitivity	0.7500	[0.594, 0.909]
Specificity	0.6061	[0.437, 0.774]
Precision / F1	0.6176 / 0.6774	—
ROC-AUC	0.7798	[0.661, 0.888]
PR-AUC	0.6839	[0.527, 0.875]

Confusion matrix: TN=20, FP=13, FN=7, TP=21 (n=61).

A second finding reported plainly: the final model, refit on all 242 rows, does *not* show the CV folds’ degenerate all-negative behavior — sensitivity is a substantive 0.75. This CV-vs-final-refit discrepancy is documented as an open, unexplained observation (a plausible but *unconfirmed* read: more training data shifts which points become support vectors), flagged explicitly for Phase 6 investigation rather than explained away.

7.6 Descriptive comparison with Phase 3 (no significance test performed, by design)

Model	Accuracy	Sensitivity	Specificity	ROC-AUC
Logistic Regression	0.869	0.857	0.879	0.921
RBF-SVM	0.869	0.893	0.848	0.924
Random Forest	0.869	0.857	0.879	0.917
XGBoost	0.836	0.857	0.818	0.928
QSVM (fidelity kernel)	0.672	0.750	0.606	0.780

All five rows use the identical 4-dimensional representation and identical locked split. **On every metric measured, the classical models outperform the QSVM on this dataset, at this**

qubit count, with this feature map. Stated as a direct numeric fact. **What this does and does not establish** (verbatim from QUANTUM_QSVM.md §10):

Establishes: the full pipeline works correctly end to end at scale; leakage-safe C-tuning on a precomputed quantum kernel is sound and verified; on this specific configuration, classical wins numerically.

Does NOT establish: that quantum kernels are inferior in general, or under a different feature map/qubit count; that the CV-tie-but-different-sensitivity pattern is understood; that the CV-vs-final-test discrepancy has a confirmed cause; anything about statistical significance (not tested); anything about real hardware (pure simulation).

8 Current Repository Structure

```
QuantumDx/
|-- PRD.md # Product Requirements Document (43 sections, never modified)
|-- MVP_SPEC.md # MVP spec + 13-phase plan (never modified)
|-- conftest.py # puts repo root on sys.path for pytest
|-- configs/
| |-- preprocessing.yaml # Phase 2 config (mirrors PreprocessingConfig)
| '-- quantum.yaml # Phase 4 config (mirrors QuantumConfig)
|-- data/
| '-- raw/ # UCI Heart Disease files; NEVER modified; checksum-verified
|-- src/
| |-- data/
| | '-- inspect_dataset.py # Phase 1
| |-- preprocessing/
| | |-- config.py # Phase 2
| | |-- pipeline.py # Phase 2 (SharedFeaturePipeline, run_preprocessing_pipeline)
| | '-- validation.py # Phase 2
| |-- classical/
| | |-- models.py # Phase 3 (4-model registry)
| | |-- tuning.py # Phase 3 (sklearn adapters, CV_SCORING, MODEL_SELECTION_METRIC)
| | |-- evaluation.py # Phase 3 (metrics, CIs, ModelResult, plots)
| | '-- train_baselines.py # Phase 3 orchestrator
| '-- quantum/ # ONLY package that imports Qiskit
| |-- config.py # Phase 4 (QuantumConfig)
| |-- data_contract.py # Phase 4 (load_quantum_dataset)
| |-- feature_maps.py # Phase 4 (zz/z/pauli_feature_map)
| |-- backends.py # Phase 4 (StatevectorBackend + stubs)
| |-- kernel.py # Phase 4 (fidelity kernel + diagnostics + cache)
| |-- sanity_experiment.py # Phase 4 orchestrator (small subsample)
| |-- qsvm.py # Phase 5 (SVC precomputed wrapper + CV)
| '-- qsvm_experiment.py # Phase 5 orchestrator (full 242/61 scale)
|-- tests/
| |-- test_dataset.py # 27 tests (Phase 1)
| |-- test_preprocessing.py # 41 tests (Phase 2)
| |-- test_classical.py # 20 tests (Phase 3)
| '-- test_quantum.py # 74 tests (Phase 4: 53, Phase 5: 21)
|-- results/
| |-- classical/ # Phase 3: 3 experiments, CSVs, JSON, 15 figures
| '-- quantum/
| |-- sanity/ # Phase 4 sanity report + kernel matrices
| '-- phase5/ # Phase 5 kernels, CV grid, final metrics, comparison, figures
'-- docs/
    |-- DATASET.md # Phase 1
    |-- PREPROCESSING.md # Phase 2
    |-- CLASSICAL_BASELINE.md # Phase 3
    |-- QUANTUM_PIPELINE.md # Phase 4
    |-- QUANTUM_QSVM.md # Phase 5
```

```
'-- latex/PROJECT_CONTEXT.tex # THIS DOCUMENT
```

Total test count as of Phase 5 completion: 162 tests, all passing, zero regressions across every phase.

8.1 Environment snapshot

Component	Version
Python	3.11.9 (Windows-10-10.0.26200)
numpy	1.26.4 (deliberately kept below 2.0 — see Phase 4)
pandas	2.3.3
scikit-learn	1.8.0
scipy	1.16.3
xgboost	3.2.0
matplotlib	3.10.1
qiskit	2.2.3
qiskit-ibm-runtime	0.44.0 (pre-installed, not yet wired up)
qiskit-aer / qiskit-machine-learning	not installed, deliberately

9 How to Run Everything (command reference)

```
# Full test suite (all phases) -- ~35-60s, run this first to confirm the
# environment is healthy before doing anything else
python -m pytest -q

# Phase-by-phase test suites
python -m pytest tests/test_dataset.py -v # Phase 1, 27 tests
python -m pytest tests/test_preprocessing.py -v # Phase 2, 41 tests
python -m pytest tests/test_classical.py -v # Phase 3, 20 tests
python -m pytest tests/test_quantum.py -v # Phase 4+5, 74 tests

# Re-run the inspection / demonstration scripts (idempotent, safe)
python -m src.data.inspect_dataset
python -m src.preprocessing.pipeline
python -m src.classical.train_baselines # ~9 minutes (3 experiments x 4 models)
python -m src.quantum.sanity_experiment # <5 seconds
python -m src.quantum.qsvm_experiment # <2 seconds (cache hit), ~1s cold
```

Before doing anything destructive, verify raw-data integrity:

```
python -c "import hashlib; print(hashlib.sha256(open('data/raw/processed.cleveland.data', 'rb').read()).hexdigest())"
# must print: a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8
```

10 Future Phases

10.1 Phase 6 — Scoped IBM Quantum Hardware Validation (next, not started)

NOT STARTED

Per the plan fixed in MVP_SPEC.md §8.5 (§1.4 above) and restated at the start of Phase 5:

1. **Verify the IBM Quantum account/plan details first** — do not assume: exact free-tier quota, session-vs-batch mode availability, which real device(s) are accessible, current queue times,

and the pinned `qiskit-ibm-runtime` primitive API (`SamplerV2` interface) for the installed version (0.44.0).

2. **Build `IBMHardwareBackend`** (currently a `NotImplementedError` stub in `src/quantum/backends.py`) implementing the same `QuantumBackend` Protocol used by `StatevectorBackend` — so `kernel.py` needs zero changes.
3. **Run the scoped validation experiment:** a fixed, seeded 30-train/10-test subsample of the same locked split (≈ 735 circuits, $\approx 3\text{--}4$ minutes of QPU time). Compute the identical kernel submatrix three ways: exact `StatevectorBackend`, a noisy simulator (may require finally installing `qiskit-aer` — re-check the numpy 2.0 compatibility question first, since Phases 1–3 may tolerate it now, or may not; verify, do not assume), and the real IBM device.
4. **Report:** element-wise kernel divergence (mean/max $|\Delta K|$) between simulator and hardware; diagonal-fidelity deviation on hardware (should be exactly 1, will not be — a ground-truth-free noise indicator); kernel-concentration statistics per backend; job provenance (device name, calibration timestamp, job IDs, QPU seconds).
5. **Do not** draw an accuracy/performance conclusion from the 10-point test subsample — explicitly a divergence measurement, not a benchmark.
6. **Never queue live during a demo.** Run once, cache to disk, serve from cache thereafter (mirrors the restartable-cache pattern already established in Phase 5).
7. Credentials via a git-ignored `.env` (with a committed `.env.example`); never commit secrets.

Two concrete, open questions from Phase 5 worth investigating alongside or before Phase 6:

- Why did `C=0.01/0.1` produce a degenerate (sensitivity=0) classifier in every CV fold, yet the full-242-row refit at the same `C` did not? (Phase 5, §7)
- Is the QSVM’s low off-diagonal kernel mean (≈ 0.09 in `K_train_train`) an early sign of kernel concentration, and would a feature-map sweep (different `reps`, entanglement, or `pauli_feature_map`) change the classical-vs-quantum gap observed in Phase 5?

10.2 Later phases (per `MVP_SPEC.md`’s 13-phase plan, not yet started)

NOT STARTED

- **VQC/QNN** — the second quantum model family named in SIH26139. Explicitly the *one pre-authorized droppable item* in the MVP plan if time runs short; QSVM alone already satisfies the problem statement’s “such as QSVM, QNN, or VQC” language.
- **Statistical significance testing** (McNemar for accuracy, DeLong or paired bootstrap for AUC) comparing QSVM vs. the Phase 3 classical baselines properly — explicitly deferred out of Phase 5 by instruction, and the natural next step once Phase 6’s hardware context exists.
- **Explainability** — SHAP for classical models; permutation importance + KernelSHAP (model-agnostic) for the QSVM, since quantum models expose no native feature attributions; a PCA loading map translating the 4 quantum components back to clinical features (already computed once in Phase 2, not yet surfaced in a dedicated explainability module).
- **Robustness / generalization suite** — repeated CV variance (already partially done via bootstrap CIs), noise-injection degradation curves, subgroup fairness slices (by sex, age band), and a cross-site check against the Hungarian/Switzerland/VA UCI files (available in `data/raw/` but never used for training, only for the Phase 1 generalization check of the inspection script itself).
- **FastAPI backend** — 9 endpoints as scoped in `MVP_SPEC.md` §14, deliberately excluding a `POST /train` endpoint (training stays CLI-driven; the API only serves stored results and predictions).

- **Dashboard** — React (or a Streamlit fallback, pre-authorized if time is short) with the 7 pages specified in `PRD.md`: Home, Dataset Analysis, Patient Prediction, Model Comparison, Explainability, Quantum Circuit/Model Info, Experiment Results.
- **Final SIH demo preparation** — code freeze, a rehearsed 10-minute run sheet (already drafted in `MVP_SPEC.md` §25.2), an offline fallback recording, and the acceptance checklist in `MVP_SPEC.md` §19.

11 Key Numbers Quick-Reference

Quantity	Value
Dataset	UCI Heart Disease, Cleveland, 303 rows, 14 columns
Raw file SHA-256	a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8
Locked split_id	e471025b07519a64
Train / Test	242 / 61 (seed 42, stratified 80/20)
Selected classical features	10 (of 17 encoded, from 13 raw)
Quantum features / qubits	4 (= <code>PreprocessingConfig.pca_n_components</code>)
Quantum encoding range	$[0, \pi]$
Feature map	<code>zz_feature_map</code> , reps=2, linear entanglement
Best classical model (Exp. 1, CV)	RBF-SVM, ROC-AUC 0.9055 ± 0.019
Best classical model (Exp. 1, test)	Logistic Regression, ROC-AUC 0.9481
Classical quantum-ready baseline (Exp. 3, mean CV)	ROC-AUC 0.858
QSVM (Phase 5, test)	ROC-AUC 0.7798 [CI 0.661, 0.888]
Total tests passing	162 / 162
Phases complete	1, 2, 3, 4, 5
Next phase	6 (scoped IBM hardware validation)

This document was generated after Phase 5's completion and review. It should be regenerated or amended after each future phase to remain a reliable handoff artifact.